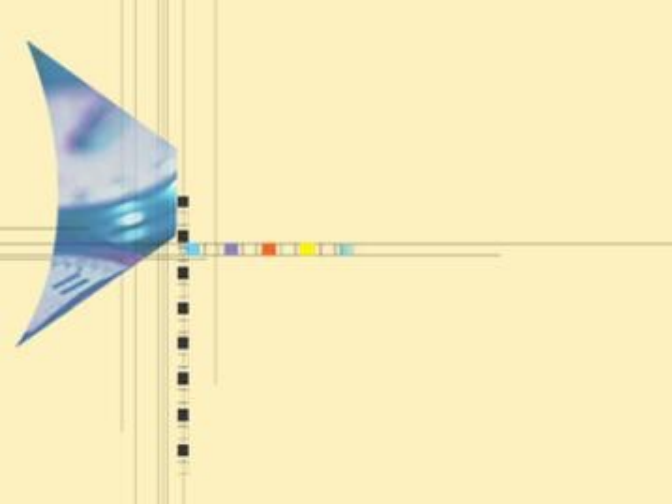




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

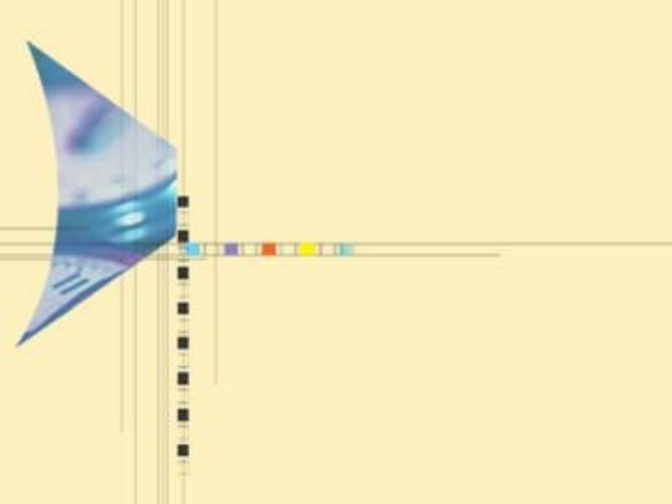
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/14

武汉大学计算机学院

武汉大学
Wuhan University





第七讲

NP完全问题

——Computability Theory



主要内容

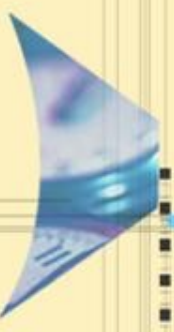
7.1 引言

7.2 判定问题

7.3 P类和NP类

7.4 多项式时间归约与NP完全性

7.5 NP完全性证明



主要内容

7.1 引言

7.1.1 图灵论题与库克论题

7.1.2 可计算的问题

7.1.2 算法复杂性与问题复杂性

7.1.4 问题的计算复杂度

7.1.5 问题的计算复杂度分析方法

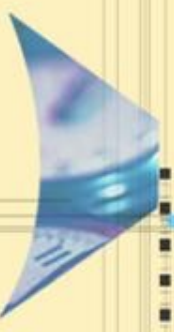
7.1.6 易解问题与难解问题

7.1.7 NP完全问题

7.1.1 图灵论题和库克论题

计算复杂性理论有两个基本论题：Church-Turing图灵论题和Cook-Karp库克论题

- Church-Turing论题：一个问题是可计算的当且仅当它在图灵机上经过有限次计算得到正确的结果。
 - 这个论题把人类所面临的问题分为两类：一类是可计算的，另一类是不可计算的。但“有限次计算”是一个宽松的条件。不可计算的含义是不论耗多少时间也不能用计算机求解。不能用计算机求解的问题称为不可计算问题或者不可解问题。
- Cook-Karp论题：一个问题实际可计算的当且仅当它在图灵机上经过多项式时间（步数）计算得到正确的结果。
 - Cook-Karp论题将可计算问题类进一步划分成两类：一类是实际可计算的，另一类是实际不可计算的或者难解的。难解问题是虽没找到多项式时间算法，但至少理论上可以用计算机求解。



7.1.2 可计算的问题

- 算法作为求解问题的方法，可以求解现实世界中很多问题。
- 但有些问题仍然无法用任何算法求解，这类问题是不可计算问题，不属于讨论的范畴。
- 我们讨论的问题都是能用计算机算法求解的问题，即可计算的问题。

7.1.2 可计算的问题

1. 首先必须是存在解的问题。

- 现实生活中有许多问题本身不存在明确的答案。例如：人们对于美的标准和喜好具有多样性，导致对于艺术作品的评价和审美体验往往具有主观性，因此，关于艺术、美学价值和审美体验的问题往往没有一个明确的答案。
- 有些问题的要求已经超出了人类的能力，可以说是无解的问题。例如“人类意识是如何产生的？”，“是否存在外星生命？”等。

2. 其次是可以通过计算机求解的问题，也就是可以用一系列机械的数学步骤（算法）来求解的问题。

- 有些问题虽然有解，但目前还不存在解决它们的明确方案，也不属于可计算问题。比如，费马小定理的证明问题，即证明一个整数 n 的阶乘的末尾数字只能为0, 1, 4, 5, 6, 9中的一个。这个问题也是有解的，但目前还没有找到一个有效的证明方法，也不属于可计算问题。
- 还有一些需要人类创造力、想象力等方面的问题，也不属于可计算问题。

7.1.3 算法复杂性与问题复杂性

- 算法的复杂性是指解决问题的一个具体的算法的执行时间，这是算法的性质；
- 问题的复杂性是指这个问题本身的复杂程度。
- 很明显，问题复杂度是问题本身固有的，算法复杂度是依赖于很多现有条件的（包括逻辑条件和物理条件，前者是设计方案能力，后者是实现能力）

7.1.4 问题的计算复杂度-以时间复杂度为例

- 问题的计算复杂度就是求解这个问题所需要的最少工作量。它是由问题本身的结构决定的内在性质，与求解它的算法好坏无关。
- 对于一个问题的求解，求解的算法（包含尚未提出的算法在内）有很多，每个算法的时间复杂度不一样，算法效率越高，占用的时间越少。求解一个问题的好的算法的时间复杂度函数应该恰好等于问题的计算复杂度。相反，一个不好的算法在执行中包含大量的冗余工作，时间复杂度函数比问题的计算复杂度可能高很多。
- 最优的算法是指求解该问题的效率最高的算法，即该算法的时间复杂度恰好与问题的计算复杂度相等，至少在阶上相等。
- 为什么要分析问题的计算复杂度？确认问题的计算复杂度与找到该问题的最优算法密切相关

最优算法-基于时间的最优性

- 含义：指求解某问题算法类中效率最高的算法
- 两种最优性

最坏情况下最优：设 A 是解某个问题的算法, 如果在解这个问题的算法类中没有其它算法在最坏情况下的时间复杂性比 A 在最坏情况下的时间复杂性低, 则称 A 是解这个问题在最坏情况下的最优算法.

平均情况下最优：设 A 是解某个问题的算法, 如果在解这个问题的算法类中没有其它算法在平均情况下的时间复杂性比 A 在平均情况下的时间复杂性低, 则称 A 是解这个问题在平均情况下的最优算法

7.1.5 问题的计算复杂度的分析方法

一般思路：通过求解某问题的算法出发来确定该问题的计算复杂度。

- 对于给定的问题，算法的时间复杂度相当于问题的计算复杂度的上界。给出一个算法，就得到一个上界。
- 随着算法的不断改进，这个上界不断降低，直到逼近问题的计算复杂度。

7.1.5 问题的计算复杂度的分析方法

如何确定上界已经降低到与问题的计算复杂度相等?

换句话讲, 如何确定正确得到解的算法所必须完成的工作量的下界---问题的计算复杂度的下界?

- 分析问题的固有性质。
 - 如, 给出针对任意求解算法设计“最坏”实例的方法, 对于这种实例统计算法至少要做多少工作, 以这个工作量作为问题计算复杂度的下界。
- 给求解问题的所有算法建立执行过程的统一模型 (如决策树), 从而对最坏情况或平均情况下的工作量进行估计, 以这个估计作为问题的计算复杂度的下界。

7.1.5 问题的计算复杂度的分析方法

- 分析方法不同，所得到的下界可能不同。
- 通过不断改进数学方法，不断提升下界，直到逼近问题的计算复杂度函数。
- 在实践中，上界和下界的改进同时进行，当上界和下界的值相等或者阶相等时，得到问题的计算复杂度。

上下界逼近确定问题计算复杂度-最坏情况为例

- (1) 设计算法 A , 求 $W(n)$, 得到算法类最坏情况下时间复杂度的一个上界
- (2) 寻找函数 $F(n)$, 使得对任何算法都存在一个规模为 n 的输入, 并且该算法在这个输入下至少要做 $F(n)$ 次基本运算, 得到该算法类最坏情况下时间复杂度的一个下界
- (3) 如果 $W(n)=F(n)$ 或 $W(n)=\Theta(F(n))$, 则 A 是最优的.
- (4) 如果 $W(n)>F(n)$, A 不是最优的或者 $F(n)$ 的下界过低.

改进 A 或设计新算法 A' 使得 $W'(n)<W(n)$.

重新证明新下界 $F'(n)$ 使得 $F'(n)>F(n)$.

重复以上两步, 最终得到 $W'(n) = F'(n)$ 或者 $W'(n) = \Theta(F'(n))$

- 至今仅有少量问题能确认问题的计算复杂度。(对很多问题, 设计高效的算法, 或给出紧凑的下界比较困难)

计算下界方法一：平凡下界

有的问题必须扫描完所有输入才能得到解，算法总要得到所有输出才能解决好，因此算法的输入规模/输出规模是它的平凡下界

例1

问题：写出所有的 n 阶排列

- 求解的时间复杂度下界为 $\Omega(n!)$ 。因为 n 阶排列共有 $n!$

例2

问题：求 n 次实系数多项式在给定 x 的值

- 求解的时间复杂度下界为 $\Omega(n)$ 。因为每个系数都要参与乘法运算，因此乘法次数不少于输入系数的个数，全部系数 n 个。

例3

问题：求两个 $n \times n$ 矩阵的乘积

求解的时间复杂度下界是 $\Omega(n^2)$ 。因为输出矩阵中有 n^2 个元素，每个元素需要至少1次乘法才能得到。

计算下界方法二：直接计数所需最少运算数

例：找最大问题：在 n 个不同的数中找最大的元素，基本运算是元素的比较。

命题：在 n 个数的数组中找最大的数，以比较做基本运算的算法类中的任何算法在最坏情况下至少要做 $n-1$ 次比较。

证明：

- 因为最大数是唯一的，其它的 $n-1$ 个数必须在比较后被淘汰。
- 一次比较至多淘汰一个数
- 所以至少需要 $n-1$ 次比较。

计算下界方法二：直接计数所需最少运算数

找最大算法Findmax

算法 Findmax

输入 数组 L , 项数 $n \geq 1$

输出 L 中的最大项 MAX

1. $MAX \leftarrow L(1); i \leftarrow 2;$
2. while $i \leq n$ do
3. if $MAX < L(i)$ then $MAX \leftarrow L(i);$
4. $i \leftarrow i + 1;$

$W(n) = n - 1$

因此，以比较作为基本运算的算法类的上界： $n - 1$

结论：Findmax 算法是最优算法。

计算下界方法三：决策树(Decision Tree)

- 决策树是一棵二叉树，对于给定问题（以比较运算作为基本运算）规定一个决策树的构造规则. 求解这个问题的不同算法所构造的决策树结构不一样.
- 决策树的每个内结点代表一次运算（如一次比较），叶结点代表一个输出（有的问题对于某些实例的输出可能在内结点）。
- 给定一个算法的决策树. 对于任何输入实例，算法将从树根开始，沿一条路径向下，在每个结点做一次基本操作(比较). 然后根据比较结果(<, =, >)走到某个儿子结点或者在该处停机. 对于给定实例的计算恰好对应了一条从树根到树叶或者某个内部结点的路径.
- 给定一个算法和输入规模 n ，对于不同的输入，算法将在对应决策树的某个结点(树叶或者内部结点)停机. 将该结点标记为一类输入. 问题输入类的数量对应于决策树的结点总数或者叶结点数.

决策树与问题复杂度

决策树的特点:

- 以比较作基本运算的算法模型
- 一个问题确定了一类决策树, 具有相同的构造规则, 该决策树类决定了求解该问题的一个算法类
- 结点数(或树叶数)等于输入分类的总数
- 最坏情况下的时间复杂度对应于决策树的深度
- 平均情况下的时间复杂度对应于决策树的平均路径长度

用决策树模型界定确定问题难度

- 给定结点数(或树叶数)的决策树的深度至少是多少?
- 给定结点数(或树叶数)的决策树的平均路径长度至少是多少?

二叉树的性质

命题1 在二叉树的 t 层至多 2^t 个结点

命题2 深度为 d 的二叉树至多 $2^{d+1}-1$ 个结点

命题3 n 个结点的二叉树的深度至少为 $\lfloor \log n \rfloor$.

命题4 设 t 为二叉树的树叶个数， d 为树深，如果树的每个内结点都有2个儿子，则 $t \leq 2^d$



例1：检索问题的决策树

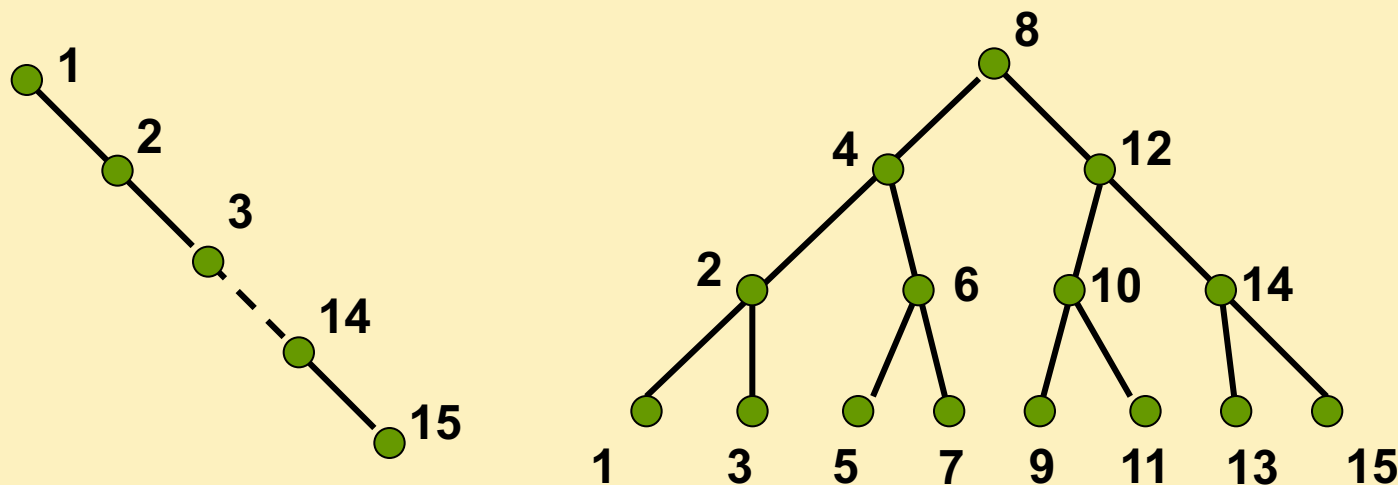
检索问题：给定按递增顺序排列的数组 L (项数 $n \geq 1$) 和数 x ，如果 x 在 L 中，输出 x 的下标；否则输出 0。

设 A 是一个检索算法，对于给定输入规模 n ， A 的一棵决策树是一棵二叉树，其结点被标记为 $1, 2, \dots, n$ ，且标记规则是：

- 根据算法 A ，首先与 x 比较的 L 中的项的下标标记为树根。
- 假设某结点被标记为 i ，分下述情况处理：
 - 当 $x < L(i)$ 时，算法 A 下一步与 x 比较的项是 $L(j)$ ，那么 i 的左儿子标记为 j ；如果算法 A 停止，则 i 没有左儿子
 - 当 $x > L(i)$ 时，算法 A 下一步与 x 比较的项是 $L(j)$ ，那么 i 的右儿子标记为 j ；如果算法 A 停止，则 i 没有右儿子
 - 若 $x = L(i)$ 时，算法 A 停止， i 没有儿子。

实例

顺序检索算法和二分检索算法的决策树, $n=15$



给定输入, 算法 A 将从根开始, 沿一条路径前进, 直到某个结点为止. 所执行的基本运算次数是这条路径的结点数. 最坏情况下的基本运算次数是树的深度+1.

检索问题的计算复杂度下界分析

定理 对于任何一个搜索算法存在某个规模为 n 的输入使得该算法至少要做 $\lfloor \log n \rfloor + 1$ 次比较.

证: n 个结点的决策树的深度 d 至少为 $\lfloor \log n \rfloor$, 故

$$W(n) = d + 1 = \lfloor \log n \rfloor + 1.$$

结论: 对于有序表搜索问题, 在以比较作为基本运算的算法类中, 二分法在最坏情况下是最优的.

例2：排序问题的决策树

考虑以比较运算作为基本运算的排序算法类

任取算法 A , 输入 $L = \{x_1, x_2, \dots, x_n\}$, 如下定义决策树:

1. A 第一次比较的元素为 x_i, x_j , 那么树根标记为 i, j
2. 假设结点 k 已经标记为 (i, j) ,

(1) $x_i < x_j$

若算法结束, k 的左儿子标记为输出;

若下一步比较元素 x_p, x_q , 那么 k 的左儿子标记为 (p, q)

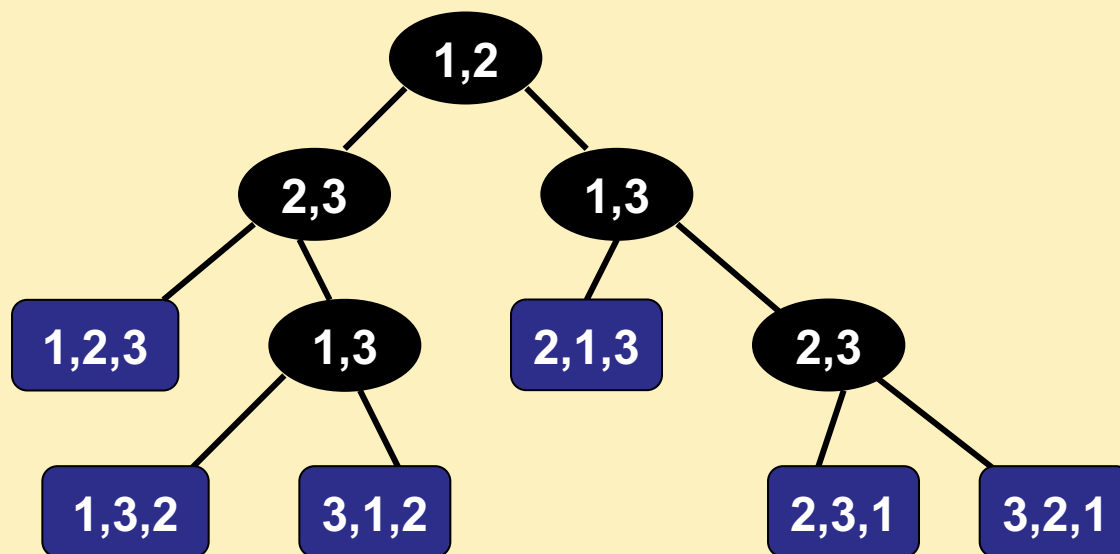
(2) $x_i > x_j$

若算法结束, k 的右儿子标记为输出;

若下一步比较元素 x_p, x_q , 那么 k 的右儿子标记为 (p, q)

实例：一棵冒泡排序的决策树

设输入为 $\{x_1, x_2, x_3\}$ ，升序排列的冒泡排序的决策树如下



任意输入：对应了决策树树中从树根到树叶的一条路径，

算法最坏情况下的比较次数：树深

不同排序算法对应的决策树结构不一样，但都有 $n!$ 片树叶

排序算法类最坏情况复杂度的下界

引理2 对于给定的 n ，任何通过比较对 n 个元素排序的算法的决策树的深度至少为 $\lceil \log n! \rceil$ 。

证明 决策树的树叶有 $n!$ 个，由二叉树性质，深度至少为 $\lceil \log n! \rceil$ 。

定理4 任何通过比较对 n 个元素排序的算法在最坏情况下的时间复杂性不低于 $\lceil \log n! \rceil$ ，近似为 $n \log n - 1.5n$ 。

$$\begin{aligned}\log n! &= \sum_{j=1}^n \log j \geq \int_1^n \log x dx = \log e \int_1^n \ln x dx \\ &= \log e (n \ln n - n + 1) \\ &= n \log n - n \log e + \log e \\ &\approx n \log n - 1.5n\end{aligned}$$

结论：堆排序、合并排序算法达到最优。

7.1.6 易求解问题与难求解问题

- 设 Π 是任意问题，如果对问题 Π 存在一个算法，它的时间复杂性是 $O(n^k)$ ，其中 n 是输入大小， k 是非负整数，我们说存在着求解问题 n 的**多项式时间算法**。这类算法在**可以接受的时间内**实现问题求解， e.g., 排序、串匹配、矩阵相乘。
- 但是，现实世界中的许多有趣问题并不属于这个范畴，因为求解这些问题所需要的时间量要用**指数和超指数函数**(如 2^n 和 $n!$) 来测度。随着问题规模的增长而**快速增长**。

n	$\log n$	n	$n \log n$	n^2	2^n
16	4 ns	0.02 ms	0.06 ms	0.26 ms	65.5 ms
32	5 ns	0.03 ms	0.16 ms	1.02 ms	4.29 sec
64	6 ns	0.06 ms	0.38 ms	4.10 ms	5.85 cent
128	0.01 ms	0.13 ms	0.90 ms	16.38 ms	10^{30} cent
512	0.01 ms	0.51 ms	4.61 ms	262.14 ms	10^{135} cent
2048	0.01 ms	2.05 ms	22.53 ms	0.01 sec	10^{598} cent
4096	0.01 ms	4.10 ms	49.15 ms	0.02 sec	10^{1214} cent
8192	0.01 ms	8.19 ms	106.50 ms	0.07 sec	10^{2447} cent
16384	0.01 ms	16.38 ms	229.38 ms	0.27 sec	10^{4913} cent
32768	0.02 ms	32.77 ms	491.52 ms	1.07 sec	10^{9845} cent
65536	0.02 ms	65.54 ms	1048.6 ms	0.07 min	10^{19709} cent

注： ns 为纳秒； ms 为毫秒； sec 为秒； min 为分钟； cent 为世纪

复杂度的阶为多项式时，算法的运行时间随输入大小 n 的增加而增加的速度非常缓慢；

而复杂度的阶为指数时，算法的运行时间随着输入大小 n 的增加呈爆炸性增加。

易求解问题与难求解问题

在学术界已达成共识：把多项式时间复杂性作为易解问题与难解问题的分界线：存在多项式时间算法的问题是易求解的（tractable），不存在多项式时间算法的问题是难求解的（intractable）。

主要原因有：

1) 多项式函数与指数函数的增长率有本质差别

问题规模 n	多项式函数					指数函数	
	$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
1	0	1	0.0	1	1	2	1
10	3.3	10	33.2	100	1000	1024	3628800
20	4.3	20	86.4	400	8000	1048376	2.4E18
50	5.6	50	282.2	2500	125000	1.0E15	3.0E64
100	6.6	100	664.4	10000	1000000	1.3E30	9.3E157

易求解问题与难求解问题

2) 计算机性能的提高对易求解问题与难解问题算法的影响

假设求解同一个问题有5个算法A1~A5，时间复杂度 $T(n)$ 如下表，假定计算机C2的速度是计算机C1的10倍。下表给出了在相同时间内不同算法能处理的问题规模情况：

$T(n)$	n (C1)	n' (C2)	变化	n'/n
$10n$	1000	10000	$n'=10n$	10
$20n$	500	5000	$n'=10n$	10
$5n\log n$	250	1842	$\sqrt{10} \ n < n' < 10n$	7.37
$2n^2$	70	223	$\sqrt{10} \ n$	3.16
2^n	13	16	$n'=n+\log 10 \approx n+3$	≈ 1

易求解问题与难求解问题

3) 多项式时间复杂性忽略了系数，不影响易解问题与难解问题的划分

问题规模n	多项式函数			指数函数	
	n^8	$10^8 n$	n^{1000}	1.1^n	$2^{0.01n}$
5	390625	5×10^8	5^{1000}	1.611	1.035
10	10^8	10^9	10^{1000}	2.594	1.072
100	10^{16}	10^{10}	10^{2000}	13780.6	2
1000	10^{24}	10^{11}	10^{3000}	2.47×10^{41}	1024

观察结论： $n \leq 100$ 时，（不自然的）多项式函数值大于指数函数值，但n充分大时，指数函数仍然超过多项式函数。

7.1.7 NP完全问题

- 前面介绍了关于搜索问题、排序问题、单源点最短路径问题等的多项式时间复杂度算法，因此它们都是易解的问题。
- 也已经证明了一些问题是难解的问题，例如生成 n 个数的全排列问题。
- 但是，人们发现现实世界还存在大量问题，如哈密顿回路问题、背包问题、货郎担问题等，截止目前还没有找到求解它们的多项式时间复杂度算法，但是也没能证明它们是难解的。由于这些问题大量存在，如何刻画这些问题的难度也是人们十分关心的问题。

7.1.7 NP完全问题

- 为了刻画目前还没有找到求解它们的多项式时间复杂度算法，但是也没能证明是难解的这些问题，重点关注一类称为NP完全的问题。
- 这一类含有许许多多问题，其中还包含了数百个著名的问题，它们有一个共同的特性：即如果它们中的一个多项式可解的，那么所有其他的问题也是多项式可解的。
- 此外，现存的求解这些问题的算法的运行时间，对于中等大小的输入也要用几百或几千年来测度。



主要内容

7.1 引言

7.2 判定问题

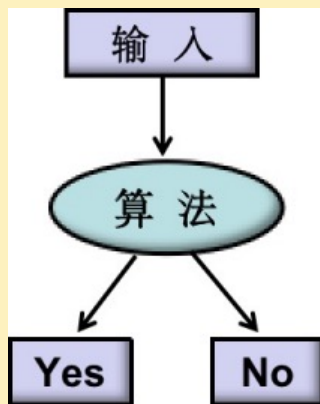
7.3 P类和NP类

7.4 多项式时间归约与NP完全性

7.5 NP完全性证明

7.2 判定问题

- 可计算问题有不同类型，如判定问题、搜索问题、优化问题、计数问题、函数问题等。
- 计算复杂性理论的研究主要限定在判定问题上。
- 判定问题：就是针对一个输入实例的输出解为“Yes”/“No”（或者“是”/“否”）的问题，也就是这类问题的答案状态只有两个：“Yes”和“No”（或者“是”和“否”）。



为什么限定为判定问题？

- 对于判定问题，可以使用布尔值来表示问题的答案，并通过判断输出的是"Yes"还是"No"来确定问题的解。这种二元性质让研究者能够较为容易地进行问题的分析和比较，因此判定问题在理论研究和分析上更为清晰和可控。
- 而对于其他类型的问题，如对于优化问题，需要确定一个最优解；而对于计数问题需要计算问题实例的数目。由于这些问题的答案不再是布尔值，给复杂性分析带来了一定的挑战。
- 因此，为便于研究，计算复杂性理论研究中将问题限定为判定问题。
- 而其它类的问题也都可以通过适当的变换转化为判定问题，且不改变问题的难易程度，从而利用计算复杂性理论进行分析和研究。

搜索问题及对应的判定问题

- 寻找多数元素问题(MAJORITY):

给定一个含有 n 个元素的数组，寻找并返回数组中的多数元素，如果数组中没有多数元素，则返回none。

- MAJORITY 问题对应的判定问题可描述如下:

多数元素判定 (MAJORITY-D) : 给定一个含有 n 个元素的数组，问数组中是否存在多数元素?

- 前面介绍过求解MAJORITY的多项式时间复杂度算法（这里记为算法A），因此，MAJORITY是易解的。
- 利用算法A很容易设计出求解MAJORITY-D的算法B：首先运行算法A，如果A返回多数元素，则B输出“Yes”；如果A返回“none”，则B输出“No”。
- 显然，B具有和A一样的算法时间复杂度。也就是说，MAJORITY-D也是易解的问题。
- 反过来，由MAJORITY-D是易解的问题也很容易得出，MAJORITY是易解的问题。

搜索问题及对应的判定问题

- **哈密顿回路搜索 (HCS)** : 给定一个无向图 G , 搜索并返回其中一条哈密顿回路, 如果没有找到哈密顿回路, 则返回“No”。
 - 所谓哈密顿回路, 是指经过图中每个顶点恰好一次的回路。如果图 G 中存在哈密顿回路, 则称 G 为哈密顿图。

HCS对应的判定问题可表述如下:

- **哈密顿回路判定 (HC)** : 给定一个无向图 G , 问 G 是哈密顿图吗?
- 假设HCS是容易求解的, 即存在一个多项式时间复杂度的算法 A 。任给一个无向图 G , 当 G 是哈密顿图时, A 输出一条哈密顿回路; 否则输出“No”。那么, 可以利用 A 按如下方式构造求解HC的算法 B : 对任给的无向图 G , 调用算法 A , 如果 A 输出 G 的一条哈密顿回路, 则 B 输出“Yes”, 如果 A 输出“No”, 则 B 也输出“No”。显然算法 B 也是多项式时间复杂度的。
 - 换句话说, 如果HCS是易求解的, 那么对应的HC也是易求解的, 这说明HCS并不比HC更容易。
 - 反过来讲, 如果HC是难求解的, 则对应的HCS也是难求解的, 这也说明HC并不比HCS更难。

优化问题及对应的判定问题

- **最长公共子序列问题 (LCS)**：给定取自相同字母集合的两个序列X和Y，求X和Y的最长公共子序列

LCS对应的判定问题可表述如下：

- **K公共子序列问题 (KCS)**：给定取自相同字母集合的两个序列X和Y 以及正整数K，问X和Y是否存在长度不小于K的公共子序列？

- 求解LCS的动态规划算法（这里记为算法A），该算法是多项式时间复杂度的算法，即LCS是易解的问题。我们利用求解LCS的多项式时间算法A很容易设计出求解KCS的算法B：首先利用算法A求出X和Y的最长公共子序列Z，如果Z的长度 $|Z| \geq K$ ，则输出“Yes”；否则，输出“No”。
- 显然，B也是多项式时间复杂度算法。也就是说，由LCS是易解的问题可知KCS也是易解的问题；反之已然。

优化问题及对应的判定问题

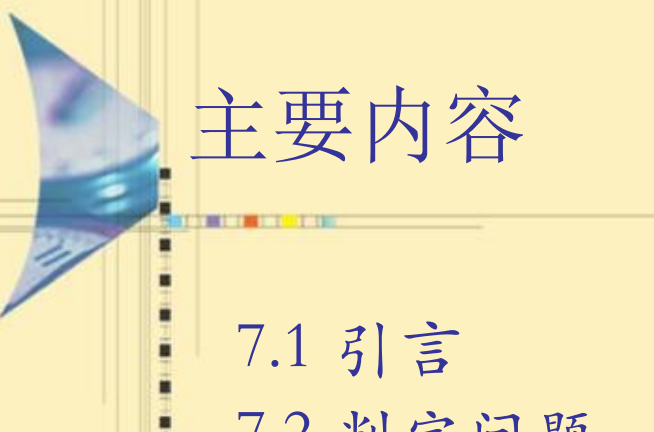
- **货郎担问题-优化 (TSP)** : 给定 n 个城市的集合, 城市 i 和城市 j 之间的距离为整数, 求一条经过每个城市恰好一次最后回到出发点的最短回路。

TSP对应的判定问题可表述如下:

- **货郎担问题-优化 (TSP-D)** : 给定 n 个城市的集合合一个正整数 K , 城市 i 和城市 j 之间的距离为整数, 问是否存在经过每个城市恰好一次最后回到出发点且长度不超过 K 的回路?
- 假设**TSP**是容易求解的, 即存在一个多项式时间复杂度的算法A。那么, 可以按如下方式构造求解**TSP-D**的算法B: 对任给的**TSP-D**的实例, 调用算法A求出最短回路, 如果这条回路的长度为 $d \leq K$, 则B输出“Yes”; 否则, B输出“No”。显然算法B也是多项式时间复杂度的。
 - 换句话说, 如果**TSP**是易求解的, 那么对应的**TSP-D**也是易求解的, 这说明**TSP**并不比**TSP-D**更容易。
 - 反过来讲, 如果**TSP-D**是难求解的, 则对应的**TSP**也是难求解的, 这也说明**TSP-D**并不比**TSP**更难。

判定问题的表示

- 如不特别说明，以后提到的问题均为判定问题。
- 为描述方便，可将判定问题 Π 形式上定义为一个有序对 $\langle D_\pi, Y_\pi \rangle$ ，其中 D_π 为 Π 的所有可能的实例组成的集合， $Y_\pi \subseteq D_\pi$ 是由 D_π 中所有答案为“Yes”的实例组成的集合。



主要内容

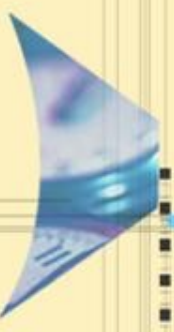
7.1 引言

7.2 判定问题

7.3 P类和NP类

7.4 多项式时间归约与NP完全性

7.5 NP完全性证明



主要内容

7.3 P类和NP类

7.3.1 图灵机模型

7.3.2 P类

7.3.3 NP类

7.3.4 P和NP关系

复杂性类

- 复杂性类 (Complexity classes) 是计算复杂性理论中用于描述问题求解难度的概念。将问题按照求解的难度分成不同的类，这就是复杂性类。每一个复杂性类是由一些具有类似求解难度的问题所组成的集合。
- 常见的复杂性类包括：
 - P类 (Polynomial Time)
 - NP类 (Nondeterministic Polynomial Time)
 - NP-完全类 (NP-Complete)
 - NP-困难类 (NP-Hard)
 - ○ ○ ○ ○

7.3.1 图灵机模型

- 图灵机是一个模拟算法运行的抽象机器，该机器根据规则表来操纵带子上的符号。尽管模型很简单，但是在任何给定计算机算法的情况下，都可以构建出模拟该算法逻辑的图灵机。
- 图灵机的逻辑结构包括：
 - 一个无限长度的带子，这个带子被分成了一个一个的单元格，每个格子有包含一个来自字母表的符号（含有空白）。
 - 一个读写头，可在纸带上左右移动，并可以读、修改格子上的符号。
 - 一个状态寄存器，用来存储图灵机的当前状态。图灵机的所有可能状态数目是有限的，并且有一个特殊的状态，称为停机状态。
 - 一个指令表（控制单元），根据机器当前所处的状态和带子上当前的符号，指示机器进行特定的操作。比如：擦除或者写入一个符号、向左或者向右移动磁头。并改变状态寄存器的值，令机器进入一个新的状态或保持状态不变。
- 可以看到整个图灵机基本上模拟了算法的执行步骤。

7.3.1 图灵机模型

- 图灵机工作时，大体重复如下过程：读写头读出当前格子的符号；根据当前状态和读到的符号找到对应的控制指令，并根据控制指令，执行以下三个动作：
 - (1) 读写头在格子上擦除或写入一个数字或符号；
 - (2) 图灵机状态转移到一个新的状态；
 - (3) 读写头向左或向右移动一格。
- 上述过程会一直重复，直到机器到达一个终止状态，或者无法继续执行操作为止。
- 图灵机的状态转移方案可以是唯一的，也可以不是唯一的。据此，图灵机模型可以分为**确定性图灵机**（Deterministic Turing Machines, DTM）和**非确定性图灵机**（Nondeterministic Turing Machines, NTM）。传统意义上所讲的图灵机模型实质就是确定性图灵机，只是一般没有特别强调“确定”罢了。

The diagram illustrates the concept of a function $f(n)$. It shows a vertical sequence of points with downward arrows, representing a sequence of values. A box labeled "Yes 或 No" is connected to the bottom point, indicating a decision or output for each input n .

确定性图灵机

- 所谓确定性图灵机 (DTM) , 就是在计算的每一时刻, 根据当前状态和读写头所读的符号, 图灵机只存在唯一的状态转移方案。
- 如上图 (左) 所示, 给DTM一个具体的问题输入实例, 它便会以一种唯一确定的方式进行运行, 并得到一个与输入对应的唯一的YES或者NO的输出结果。
- 针对某个输入, 如果DTM机输出YES, 就表示接受该输入; 如果输出NO, 则表示拒绝该输入。

确定算法

- 可以用DTM模型描述的算法，称之为确定性算法。
- 确定性算法的非形式化定义：
 - 设A是求解问题 Π 的一个算法，如果在展示问题 Π 的一个实例时，在整个执行过程中，每一步都只有一种选择，则称A是确定性算法。因此如果对于同样的输入，实例一遍又一遍地执行，它的输出从不改变。
 - 如果A的时间是关于输入实例大小的多项式时间，则称A是多项式时间确定性算法。

非确定性图灵机

- 非确定性图灵机 (NTM) 是DTM的一种推广, NTM和DTM的不同之处在于, 在计算的每一时刻, 根据当前状态和读写头所读的符号, NTM存在多种状态转移方案, 机器可以任意选择其中一种方案继续运行, 直至停机。
- 如上图 (右) 所示, NTM的问题求解路线类似一棵计算树, 树中某些节点存在多个分支, 在一次具体的执行步骤中, 具体选择哪个分支是随机的, 不确定的。这就意味着, 针对相同的输入实例, NTM的不同次执行结果可能会不同, 既有可能输出YES, 也有可能输出NO。
- NTM的工作机制可以看作包含“随机猜测”(从所有可能的转移状态中随机猜测一个) 和确定性的验证 (对猜测结果的验证) 两个环节。
- 在NTM的所有可能输出中, 只要有一次输出YES, 就说NTM接受该输入; 如果全部可能的输出结果都是NO, 则表示NTM拒绝该输入。

不确定算法

- 可以用NTM模型描述的算法，称之为不确定性算法。

不确定性算法的非形式化定义：

- 设A是求解问题 Π 的一个算法，如果算法A以如下**猜测+验证的方式工作**，称算法A为不确定性(nondeterminism)算法：
 - **猜测阶段**：对问题的输入实例x产生一个任意字符串y，在算法的每次运行，y可能不同，因此猜测是以不确定的形式工作。
 - **验证阶段**：在这个阶段，用一个确定性算法验证两件事：首先验证猜测的y是否是合适的形式，若不是，则算法停下并回答no；若是合适形式，则继续检查它是否是x的解，如果确实是x的解，则停下并回答yes，否则停下并回答no。。
- 如果猜测和检查都在输入实例的多项式时间完成，则称A为多项式时间不确定性算法。
- 注意：若A输出no，并不意味着不存在一个满足要求的解，也就是说，并不意味着A不接受该输入，因为猜测可能不正确。

7.3.2 P类

- **定义：**判定问题的P类由这样的判定问题组成，它们的 *yes/no* 解可以用确定性算法在运行多项式步数内，例如在 $O(n^k)$ 步内得到，其中 k 是某个非负整数， n 是输入大小。也就是说：如果对于某个判定问题 Π ，存在一个非负整数 k ，对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到 *yes* 或 *no* 的答案，则称该判定问题 Π 是一个 **P(Polynomial) 类问题**。
- 根据定义，对一个判定问题，如能构造一个多项式求解算法，则可证明该问题属于P类。
- 显然，所有易解问题都是P类问题。

P类问题的例子

排序问题:

给出一个 n 个整数的表, 它们是否按非降序排列?

不相交集问题:

给出两个整数集合, 它们的交集是否为空?

最短路径问题:

给出一个边上有正权的有向图 $G = (V, E)$, 两个特异的顶点 $s, t \in V$ 和一个正整数 k , 在 s 到 t 间是否存在一条路径, 它的长度最多是 k 。

2着色问题:

给出一个无向图 G , 它是否是2可着色的?即它的顶点是否可仅用两种颜色着色, 使两个邻接顶点不会分配相同的颜色?注意, 当且仅当 G 是二分图, 即当且仅当它不包含奇数长的回路时, 它是2可着色的。

2可满足问题:

给出一个合取范式(CNF)形式的布尔表达式 f , 这里每个子句恰好由两个文字组成, 问 f 是可满足的吗?

P类问题性质

- 如果对于任意问题 $\Pi \in C$, Π 的补也在 C 中, 我们说问题类 C 在补运算下是封闭的。
 - 例如, 2着色问题的补可以陈述如下: 给出一个图 G , 它是不2可着色的吗? 我们称这个问题为 NOT-2-COLOR 问题。
 - 很容易证明, NOT-2-COLOR 是属于 P 的。
- 定理: P 类问题在补运算下是封闭的。

7.3.3 NP类

- P类问题是易解的问题。也已经证明一些问题是难解的问题。例如， n 个数的全部 $n!$ 个排列的生成问题的算法至少需要 $n!$ （超指数）的时间。
- 但是，人们发现现实世界还存在大量问题，截止目前还没有找到求解它们的多项式时间复杂度算法，但是也没能证明它们是难解的。
- 由于这些问题大量存在，如何刻画这些问题的难度也是人们十分关心的问题。为了刻画这一大类问题的求解难度，引入NP类的定义。

7.3.3 NP类

- **定义：**判定问题类NP由这样的问题 Π 组成：对于这些问题存在一个确定性算法A，该算法在对 Π 的一个实例展示一个断言解时，它能在**多项式时间内验证解的正确性**。即如果断言解导致答案是yes，就存在一种方法可以在多项式时间内**验证这个解**。
- **定义：**判定问题类NP由这样的判定问题组成：对于它们存在着**多项式时间内运行的不确定性算法**。也就是说，如果对于判定问题 Π ，存在一个非负整数k，对于输入规模为n的实例，能够以 **$O(n^k)$ 的时间运行一个不确定性算法，得到yes/no的答案**，则该判断问题 Π 是一个NP(**nondeterministic polynomial**)类问题。

NP类问题例子1

- 考虑问题COLORING，我们用两种方法证明这个问题属于NP类。
- 方法1：建立多项式时间确定性验证算法
 - 设I是COLORING问题的一个实例，s被宣称为I的解。
 - 容易建立一个确定性算法来验证s是否确实是I的解。
 - 从NP类的非形式定义可得COLORING问题属于NP类。

NP类问题例子1

方法2: 建立不确定性算法

- 当图G用编码表示后，一个算法A可以很容易地构建并运行如下：
 - 首先通过对顶点集合产生一个任意的颜色指派以“猜测”一个解。
 - 接着，A验证这个指派是否是有效的指派，如果它是一个有效的指派，那么A停下并且回答yes，否则它停下并回答no。
 - 请注意，根据不确定性算法的定义，仅当对问题的实例回答是yes时，A回答yes。
 - 其次是关于需要的运行时间，A在猜测和验证两个阶段总共花费不多于多项式时间。

NP类问题例子2

哈密顿回路问题是NP类问题

证明1: 可设计如下多项式不确定性求解算法：对任意给定的一个无向图 G ，随机猜测 G 中所有顶点的一个排列，然后检查这个排列是否构成一条哈密顿回路，即相邻的两个顶点之间是否有边以及排列的首尾顶点之间是否有边。如果是，则回答“Yes”，否则回答“No”。当 G 是哈密顿图时，总能猜对一个顶点的排列，它确实给出一条哈密顿回路，从而使算法回答“Yes”；如果 G 不是哈密顿图，则猜测的任何排列都不是哈密顿回路，从而算法总是回答“No”。显然，猜测一个排列和检查这个排列是否能表示哈密顿回路都可以在多项式时间内完成。因此，哈密顿回路问题是NP类问题。任何一个构成哈密顿回路的顶点排列都是 G 为哈密顿图的证据。

NP类问题例子2

哈密顿回路问题是NP类问题

证明2: 可设计如下多项式确定性验证算法：对任意给定的一个无向图 G 和一个声称是 G 的一条哈密顿回路 L ，检查 L 是否真的是一条哈密顿回路。首先检查 L 是否遍历 G 中每个顶点且只遍历1次，如果不是，回答“No”；如果是，继续检查 L 中相邻的两个顶点之间是否在 G 中有边，如果是，则回答“Yes”，否则回答“No”。当 G 是哈密顿图时，一定会有一条哈密顿回路 L 是图 G 为哈密顿图的证据，从而使算法回答“Yes”。显然，检查 L 是否是哈密顿回路可以在多项式时间内完成，即哈密顿回路问题是多项式时间可验证的。因此，哈密顿回路问题是NP类问题。

NP类对补运算封闭吗?

- 前面已经介绍, P 类对求补运算是封闭的, NP 类对求补运算是否是封闭的呢? 尽管人们做类大量工作, 但还是没有人知道 NP 类在补运算下是否是封闭的, 即 $\Pi \in NP$ 是否说明 $\bar{\Pi} \in NP$?
- 复杂类性 $co - NP$ 是由所有满足 $\bar{\Pi} \in NP$ 的问题 Π 所组成的集合。 NP 类在补运算下是否是封闭的问题就可重新表述为是否有 $NP = co - NP$?

NP类对补运算封闭吗？

- 下面例子似乎支持 $NP \neq co-NP$ ，但迄今也没有得到严格的理论证实。
 - 例如：货郎担问题的补问题可描述为：给定 n 个城市和它们之间的距离，是否不存在长度不超过 K 的任何周游路线？似乎不存在一个多项式不确定性算法能不穷尽所有 $(n-1)!$ 的可能性来求解这个问题。
 - 再例如，可满足性问题的补问题：给定一个含有 n 个变元的布尔表达式 f ， f 是不可满足的吗？即不存在真值指派使 f 为真。这个问题似乎也不存在一个多项式的不确定性算法可以不检查所有 2^n 种指派而求解这个问题。

7.3.4 类P和类NP关系

例1: 在一个周六的晚上，你参加了一个盛大的晚会。由于感到局促不安，你想知道这一大厅中是否有你已经认识的人。宴会的主人向你提议说，你一定认识那位正在甜点盘附近角落的A。不费一秒钟，你就能向那里扫视，并且发现宴会的主人是正确的。然而，如果没有这样的暗示，你就必须环顾整个大厅，一个个地审视每一个人，看是否有你认识的人。

例2: 如果某人告诉你，数13717421可以写成两个较小的数的乘积，你可能不知道是否应该相信他，但是如果他告诉你它可以分解为3607乘上3803，那么你就可以用一个袖珍计算器容易验证这是对的。

类P和类NP关系

- P类问题就是易求解的问题，NP类问题就是易验证的问题。
- 根据经验，如果能从头解决一个问题，那么也一定能验证一个已明确给出的解是否正确，反之则不一定。生成问题的一个解通常比验证一个给定的解容易得多。

这个经验类比到P类和NP类上，可以得出如下结论：

$$P \subseteq NP。 \text{（证明略）}$$

- 目前还不知道是否有 $P = NP$ ，但大多数从事理论研究的计算机科学家认为P和NP不是一个问题类，研究者们倾向认为NP包含一些不属于P类的问题，即 $P \neq NP$ ，但目前还没有令人信服的理论研究成果。

P、NP、co-NP关系

- 由于 P 类在补运算下是封闭的，很容易证明 $P \subseteq co - NP$ 。
- 因此可以明确的关系： $P \subseteq NP$ ， $P \subseteq co - NP$ ，即 $P \subseteq NP \cap co - NP$ ；
- 不能明确的关系：不知道是否有 $P = NP \cap co - NP$ ，或者 $NP \cap co - NP$ 中是否还存在不是 P 类的问题。
- 大部分学者认为复杂类 P 、 NP 和 $co - NP$ 之间的关系如下图：

